

#30 · Redis 快取

[未來] 技術代號 D-P1-1 · 約 1-2 週

一句話：從個人筆記本搬到公共筆記本

為什麼要修

快取放在單台伺服器記憶體 (in-process LRU 1000 條 TTL 300s) 。

現在怎樣

server/_core/cache.ts CacheService 內部 LRU。multi-instance 部署時每台各自快取、不一致。

問題在哪

未來擴張多台伺服器時浪費 + 不一致。

要改什麼

把 cache.ts 抽象為 interface、新增 RedisAdapter、graceful degrade 到 in-memory；灰度遷移：5% → llm:* → search:* → prefs:*

涉及檔案

server/_core/cache.ts (抽 interface)

server/_core/cacheAdapters/RedisAdapter.ts (新增)

server/_core/cacheAdapters/LruAdapter.ts (從原檔拆出)

Dockerfile / docker-compose / railway 加 Redis service

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 C.2 實作 D-P1-1 Redis 分散式快取：

何時做：當你準備擴張到多台伺服器同時跑的時候做。在此之前沒迫切需求。預估 1-2 週。

【第 1 部分：開 Redis】

1. 在你的雲端開 Redis (建議 Upstash / Railway Redis / AWS ElastiCache)。
2. 設定 maxmemory-policy allkeys-lru。
3. 新增 env var REDIS_URL。

【第 2 部分：抽象介面】

4. 修改 server/_core/cache.ts，把 CacheService 抽為 interface：

```
```typescript
export interface CacheService {
 get<T>(key: string): Promise<T | null>;
 set<T>(key: string, val: T, opts?: CacheSetOptions): Promise<void>;
 delete(key: string): Promise<void>;
 deleteByPrefix(prefix: string): Promise<void>;
}
```
```

5. 把既有 LRU 實作搬到 server/_core/cacheAdapters/LruAdapter.ts。

【第 3 部分：Redis adapter】

6. 新增 server/_core/cacheAdapters/RedisAdapter.ts：

```
```typescript
import Redis from "ioredis";

export class RedisAdapter implements CacheService {
 private client: Redis;

 constructor(url: string) { this.client = new Redis(url, { maxRetriesPerRequest: 2 }); }

 async get<T>(key: string) { const v = await this.client.get(key); return v ? JSON.parse(v) : null; }
 async set<T>(key, val, opts) { await this.client.setex(key, opts?.ttl ?? 300, JSON.stringify(val)); }
 async delete(key) { await this.client.del(key); }
 async deleteByPrefix(prefix) {
 const keys = await this.client.keys(`${prefix}*`);
 if (keys.length) await this.client.del(...keys);
 }
}
```

```
}
}
...
```

#### 【第 4 部分：工廠 + graceful degrade】

7. 在 cache.ts 頂部：

```
```typescript  
export function initCache(): CacheService {  
  if (serverEnv.REDIS_URL) {  
    try { return new RedisAdapter(serverEnv.REDIS_URL); }  
    catch (err) { logger.warn("redis_init_failed_fallback_lru", { err }); }  
  }  
  return new LruAdapter();  
}  
...`
```

【第 5 部分：灰度遷移】

8. Feature flag CACHE_REDIS_TRAFFIC_PERCENT (0-100，預設 5)：5% 流量試 Redis、95% 走 LRU。

9. 監控 p95 延遲、命中率 7 天。

10. 階段升級：llm:* (最貴) → search:* → prefs:*

11. 同時設「LRU 為永遠 L1、Redis 為 L2」旁路非強依賴。

【第 6 部分：依賴】

12. npm install ioredis

不要做的事：不要把所有 in-memory state 都改 Redis (只 cache)。不要對 collaboration session 用 Redis (屬 DB 該做的事，已在第 3 條完成)。不要設無 TTL key (會 memory leak)。

如何驗證做完了

REDIS_URL 設了 → 重啟伺服器 → metrics 看到 cache.backend=redis。連跑兩次相同 LLM prompt → 第二次 < 50ms。手動 stop Redis → 應降級到 LRU、無 crash。

預估耗時

1-2 週